

SEARCHING

Q2)

Question:

DS	Session	Searching	Question Information	Level 1	Challenge 2
Problem Description: King Alexander wants every chariot to line up for the start of her Winter-eve ball. He has asked you, Twilight Sparkle, to sort the horse chariots alphabetically but with royalty in front. Royal horses chariot have diamonds in their names. Constraints: 1 <= Names <= 100 Input Format: list of horse chariots numbering anywhere from 1 to 40 horse chariot, with 1 per line. The end of input will be marked with END on a line by itself. Names should be no longer than 100 characters in length, and to only contain letters and spaces. Output Format: Print the output in a separate lines contains Sort and list the horse chariots alphabetically in ascending order ('A' "first"), ignoring case. However, any horse chariot with a diamond in their name must be placed at the "top" of the list (before the "A's" start) in the diamond order given in the Discussion section below. Explanation: The Pony ranking (in Ascending order) for gemstones is a follows: Lapis, Topaz, Tourmaline, Sapphire, Peridot, Ruby, Pearl, Emerald, Diamond, Aquamarine, Amethyst, Garnet. you are guaranteed that you will not have to deal with any gemstones not listed above. If multiple gems are listed in the same name, sort by whichever gem has "highest prioity" (e.g. a pony named 'Garnet Lapis' would be listed before a pony named 'Topaz Sapphire,' because Lapis has the highest priority). In the case of equal ranking on gemstones, output in alpha order by total name (not just the gemstones), only sort by highest priority gemstones, after that by alpha. (Example, given the names: Lapis Topaz and Amethyst Lapis, they should be printed in order as: Amethyst Lapis then Lapis Topaz. That is because both names have the highest ranked gemstone (Lapis), so we simply sort them in ascending order alphabetically after their ranking has been established. We would not list Lapis Topaz before Amethyst Lapis because Topaz has a higher ranking over Amethyst. Stop comparing gemstone ranking after determine the highest rank of the gems in the name.) You are guaranteed that there will be no ties in priority for gemstone ranking. You are also guaranteed that you will not encounter hyphenated names like Ruby-Sue. However, if a gemstone name happened to be found as part of a name (like Rubyanne) you can safely treat that as just another name to alphabetise. Gemstone names have to stand on their own					
(separated by spaces, or the entirety of the name) to be treated royally.					

PROGRAM:

```
1  #include <stdio.h>
2  #include <string.h>
3  #include <ctype.h>
4
5  char gem[10] = {"Lapis", "Topaz", "Tourmaline", "Sapphire", "Peridot", "Ruby", "Pearl", "Emerald", "Diamond", "Aquamarine", "Amethyst", "Garnet"};
6
7  int rank(char *s) {
8      char t[100], *p;
9      strcpy(t, s);
10     p = strtok(t, " \n");
11
12     while (p) {
13         for (int i = 0; i < 12; i++) {
14             int j = 0;
15             while (p[j] && gem[i][j] && tolower(p[j]) == tolower(gem[i][j])) j++;
16             if (!p[j] && !gem[i][j]) return i;
17         }
18         p = strtok(NULL, " \n");
19     }
20     return 12;
21 }
22
23 int cmp(char *a, char *b) {
24     int x = rank(a), y = rank(b);
25
26     if (x != y) return x - y;
27
28     for (int i = 0; a[i] || b[i]; i++) {
29         char c1 = tolower(a[i]), c2 = tolower(b[i]);
30         if (c1 != c2) return c1 - c2;
31     }
32     return 0;
33 }
34
35 int main() {
36     char a[100][105], t[105];
37     int n = 0;
38
39     while (fgets(a[n], 105, stdin)) {
40         a[n][strlen(a[n])] = 0;
41         if (strcmp(a[n], "END") == 0) break;
42         n++;
43     }
44
45     for (int i = 0; i < n - 1; i++)
46         for (int j = i + 1; j < n; j++)
47             if (cmp(a[i], a[j]) > 0) {
48                 strcpy(t, a[i]);
49                 strcpy(a[i], a[j]);
50                 strcpy(a[j], t);
51             }
52
53     printf("\n");
54
55     for (int i = 0; i < n; i++)
56         printf("%s\n", a[i]);
57
58     return 0;
59 }
```

OUTPUT:

TEST CASE 1:

```
Buttershy  
Orangejack  
Rubyanne  
Rainbow Dash  
Rarity  
Pinkie Pie  
Emerald Sunshine  
Spike the Dragon  
Misty Sapphire  
END
```

```
Misty Sapphire  
Emerald Sunshine  
Buttershy  
Orangejack  
Pinkie Pie  
Rainbow Dash  
Rarity  
Rubyanne  
Spike the Dragon
```

TEST CASE 2:

```
Granny Smith  
Discord  
Spitfire  
Granny Smith  
Photo Finish  
Pearl Peridot  
Granny Smith  
Princess Pearl  
Soarine  
END
```

```
Pearl Peridot  
Princess Pearl  
Discord  
Granny Smith  
Granny Smith  
Granny Smith  
Photo Finish  
Soarine  
Spitfire
```